

Bagging & Boosting

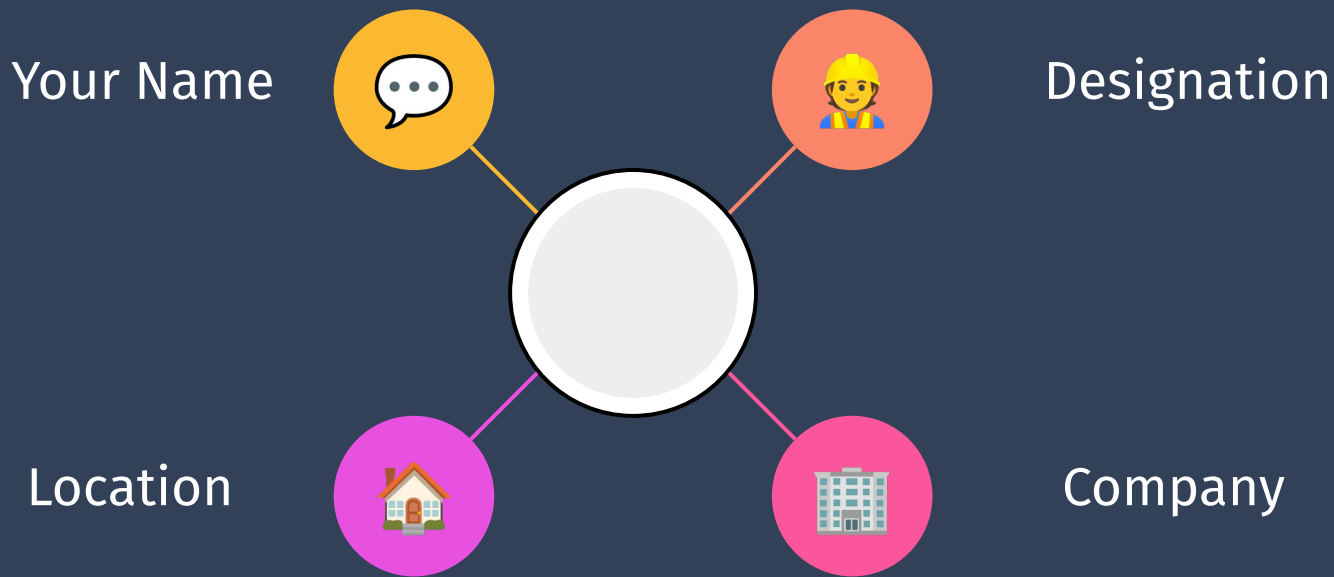
Introduction – Julie Wilkins

Senior Product Manager, Amazon
(Corporate Projects, Logistics)
MBA – Temple University
BA – Louisiana State University

Replace it with
Instructor
Name/background/rel
ated photographs to
introduce to learners



Welcome to today's class, before we begin, pop into the chat:



Structure of Courses

1. Pre-class video(s)
2. Sunday Class - 4 hr
3. Post-class-video(s)
4. Assignments
5. MCQs



Wednesday and Thursday Sessions

After the main class watch the **post class videos**

Wednesday - Technical Coaching Session

One hour of topical review on the class. This is an excellent time to reinforce Sunday's content or ask advanced questions, depending on your experience.

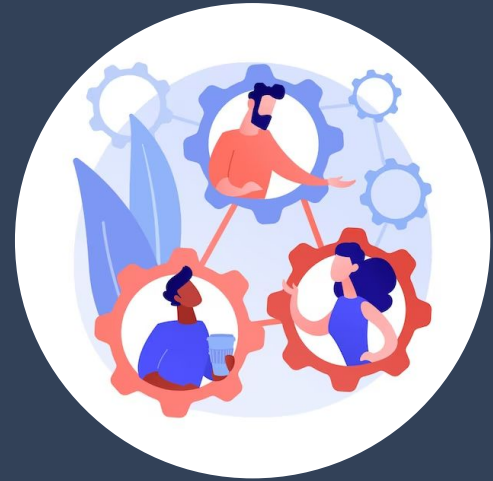
Thursday - Assignment Review Session

Two hours to review the assignment and the MCQs. Both assignments and MCQs are designed to reinforce the learning objectives which tie directly to knowledge and skills you will need in industry and in interviewing.



Optimize Your Experience

- ✓ Interact with your instructors via Sunday live class.
- ✓ Don't be shy to speak up and get clarifications !



- ✓ Solve your assignments, MCQs and other assessments and get feedback (Thursday review session)
- ✓ Use your resources! It's your experience – what you put in is what you'll get out.

Are you using all these IK support features?

- **Uplevel** - contains all the learning resources - videos, MCQs, assignments
- Optional **Post class videos** for additional reference
- Optional **Math reference videos** by Omkar Deshpande to decode the Math behind ML in detail
- **Technical coaching sessions** on Wednesday to clarify all your queries

Are you using all these IK support features?

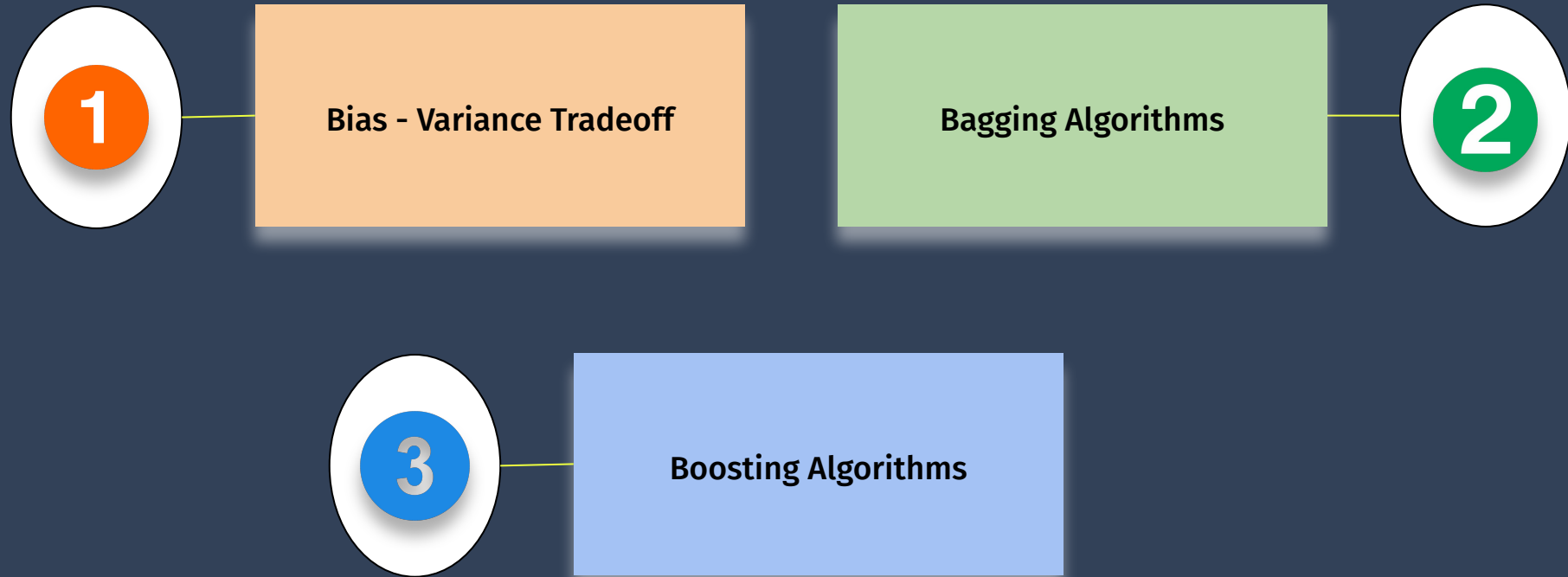
- **Discord** community for peer learning
- **TAs** to help answer the queries in the live class
- Use **academic support** for any support/advice you want
- Use **expert connect** sessions to connect to an expert for personalized guidance
- **Support tickets** - Raise them any time you want any clarifications/ tech concerns , etc

Success Hacks

- Be prepared for the class by watching the pre class videos
- Attend every class; **Sunday and Thursday**
- Participate in class
- Attempt all the assignments, mini-projects and capstone
- Actively advocate for yourself
- Keep your goal in mind
- **Consistency is the key**

Be patient with yourself !

I will help you learn these in this week's sessions...



Quick Recap

Derivatives and differentiation

Derivatives measure the rate at which a function changes concerning its input variable.

Derivative represents the slope of the tangent line to the curve at that point

Points where the derivative is zero or undefined help identify extrema or inflection.

Multivariate calculus

Loss function quantifies how well a model's predictions match the actual target values.

Partial derivative measures a function's changes w.r.t. a variable while others constant.

Gradient is a vector of partial derivatives, indicating the steepest ascent of a function

Applications of derivatives

Gradient descent is an optimization algorithm to minimize loss by adjusting its parameters.

1: Initialize parameter(s)
2: Compute the gradient
3: Update the parameters 4.
Repeat steps 2 and 3

$$\theta := \theta - \alpha \cdot \nabla J(\theta)$$



Recap Quiz

Which of the following statements best describes Stochastic Gradient Descent (SGD)?

- A** SGD computes the gradient using the entire dataset at each iteration.
- B** SGD updates the model parameters after processing all mini-batches of data.
- C** SGD updates the model parameters after computing the gradient of a single training example.

Recap Quiz

Which of the following statements best describes Stochastic Gradient Descent (SGD)?

A

SGD computes the gradient using the entire dataset at each iteration.

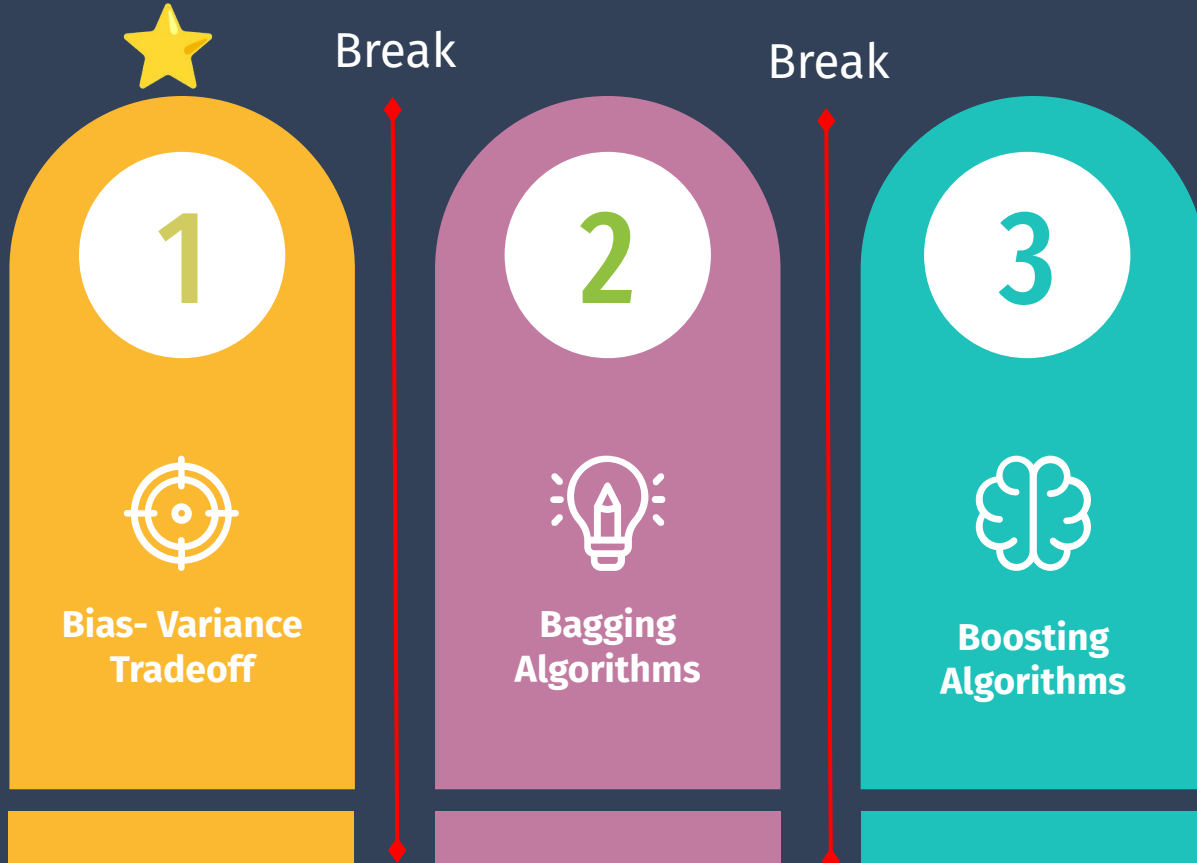
B

SGD updates the model parameters after processing all mini-batches of data.

C

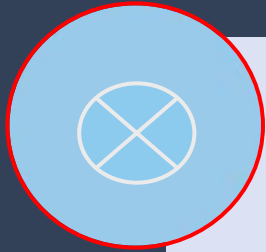
SGD updates the model parameters after computing the gradient of a single training example.

Today's Agenda

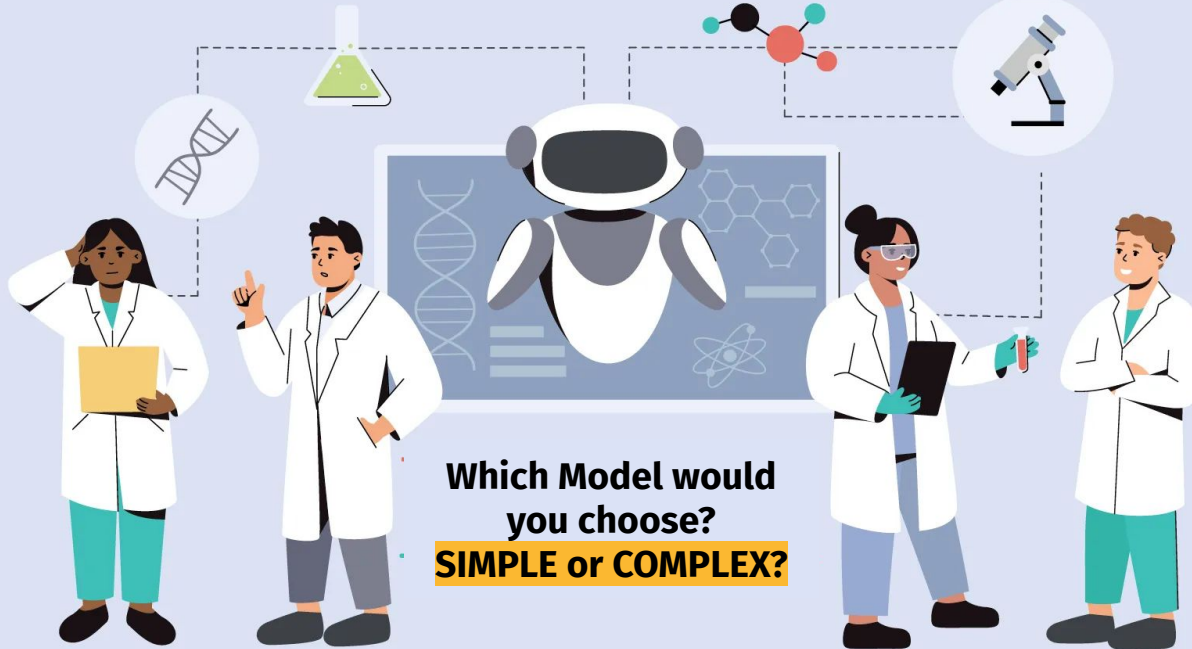
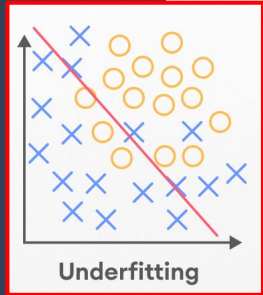


Bias - Variance Tradeoff

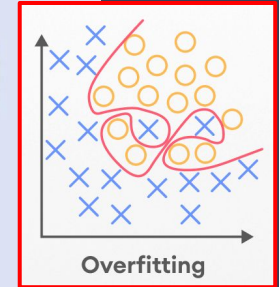
Which model to choose? Simple or Complex?



SIMPLE MODEL



COMPLEX MODEL



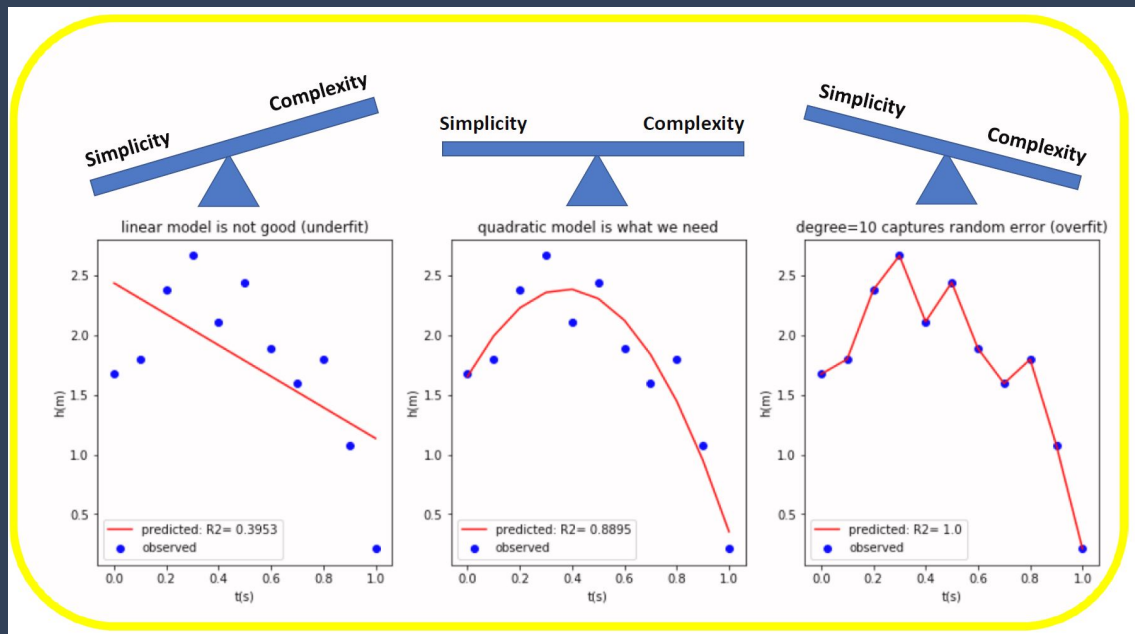
Simple vs Complex Models

The dilemma: simple vs complex models

	Explainable	Accurate
Simple model		
Complex model		

Too Simple vs Too Complex Models

Underfitting: the model didn't gather enough information from the training set, and doesn't capture the link between the features x and the target y .



Overfitting: the model is too closely related to the examples in the training set and doesn't generalize well to other examples.

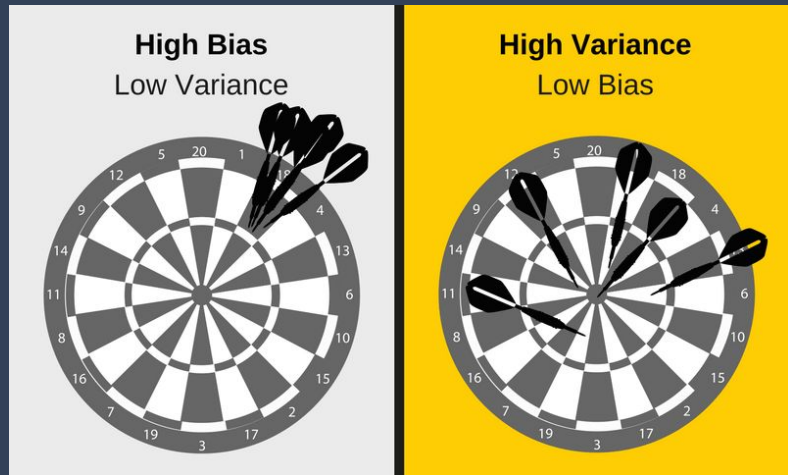
Introducing Bias-Variance

Bias: Represents the error from overly simplistic models.

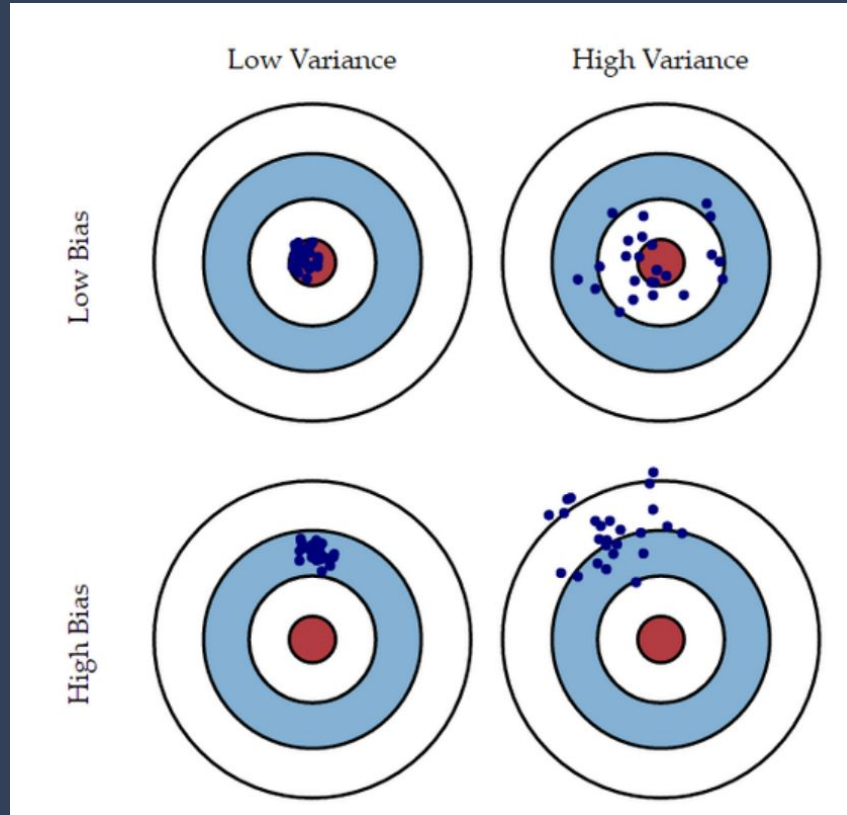
High bias leads to underfitting, where the model consistently misses important patterns in the data.

Variance: Represents the error from overly complex models.

High variance leads to overfitting, where the model is too attuned to the training data and fails to generalize to new cases.



Bias Variance Decomposition



Bias-Variance decomposition

- **Why do we do Bias-Variance decomposition?**
- Consider an ML model F applied to some test dataset
- For instance, evaluating a trained linear regression model on a test dataset.
- We compute MSE and it seems to be high → Very common
- **How to debug this problem, what goes wrong?**
- Bias Variance decomposition allows us to debug the issue and see what we should do next to reduce the error.

Bias-Variance decomposition

Our intuition can be formalized by the Bias-Variance tradeoff.

- Assume that the points in your training/test set are all taken from a similar distribution with

$$y = f(x_i) + e_i \text{ and } E(e_i) = 0 \text{ and } \text{var}(e_i) = \sigma^2$$

- Our goal is to find f and using our training set, we obtain an estimate $\hat{f}$

Bias-Variance decomposition

$$\begin{aligned}\text{Test MSE} &= \mathbb{E} \left((y - \hat{f}(x))^2 \right) \\ &= \mathbb{E} \left((\epsilon + f(x) - \hat{f}(x))^2 \right) \\ &= \mathbb{E}(\epsilon^2) + \mathbb{E} \left((f(x) - \hat{f}(x))^2 \right)\end{aligned}$$

Bias-Variance decomposition

$$\begin{aligned}\mathbb{E}[(f(x) - \hat{f}(x))^2] &= \mathbb{E} \left[\left((f(x) - \mathbb{E}[\hat{f}(x)]) - (\hat{f}(x) - \mathbb{E}[\hat{f}(x)]) \right)^2 \right] \\ &= \mathbb{E} \left[\left(\mathbb{E}[\hat{f}(x)] - f(x) \right)^2 \right] + \mathbb{E} \left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)] \right)^2 \right] \\ &\quad - 2\mathbb{E} \left[\left(f(x) - \mathbb{E}[\hat{f}(x)] \right) \left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)] \right) \right] \\ &= \underbrace{\left(\mathbb{E}[\hat{f}(x)] - f(x) \right)^2}_{=\text{bias}[\hat{f}(x)]} + \underbrace{\mathbb{E} \left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)] \right)^2 \right]}_{=\text{var}(\hat{f}(x))}\end{aligned}$$

Bias-Variance decomposition

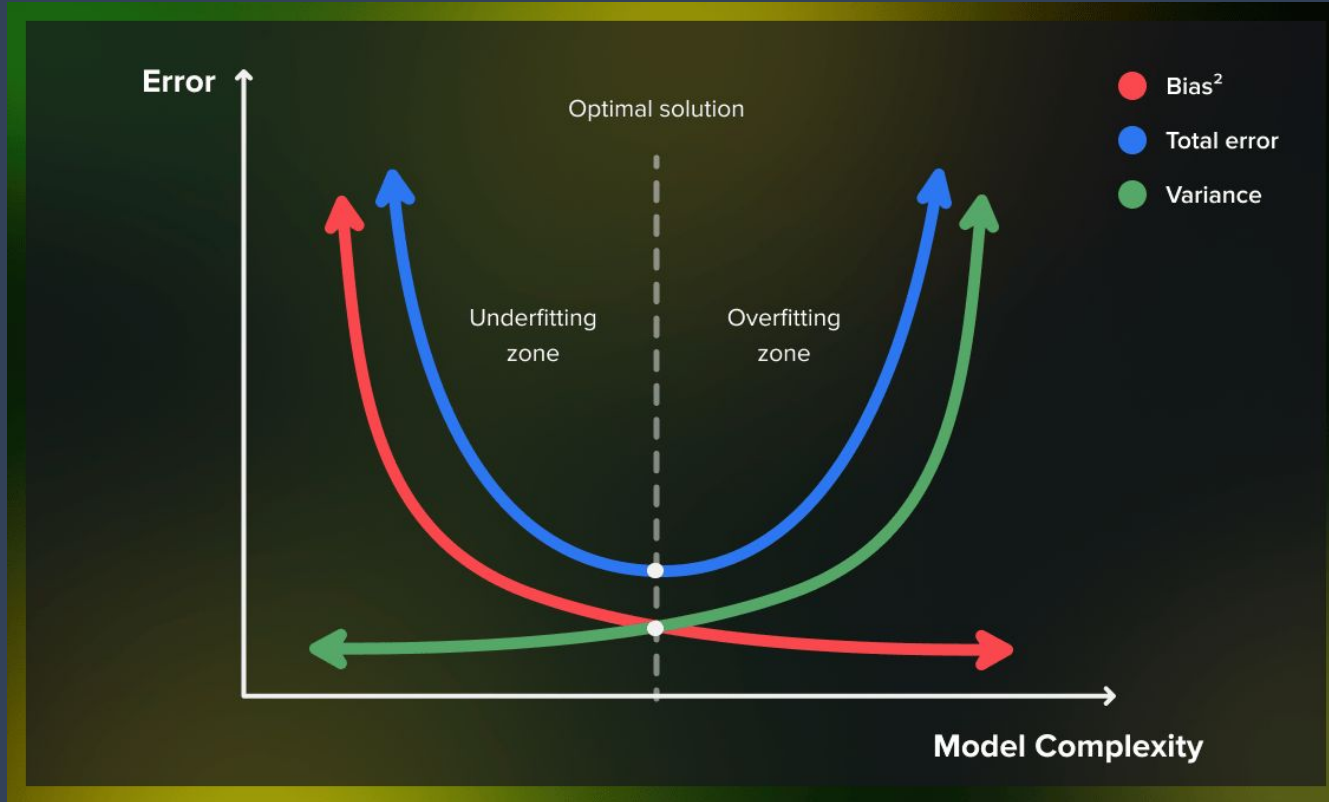
Bias/variance decomposes the expected loss into three terms:

1. **bias**: how wrong the expected prediction is (corresponds to underfitting)
2. **variance**: the amount of variability in the predictions (corresponds to overfitting)
3. **Bayes error**: the inherent unpredictability of the targets or noise in data

Bias-Variance Trade-Off

- This analysis was for regression models and used MSE as the loss function.
- We often loosely use “bias” and “variance” as synonyms for “underfitting” and “overfitting” for any ML model.
- Inherent trade-off between noise and variance.
- Our goal while training a ML model is
 - Reduce bias without increasing variance?
 - Reduce variance without increasing bias?

Bias-Variance Tradeoff



Bias Variance Trade-off

Q: High Bias or High Variance?

- KNN with large k
- KNN with $k = 1$ (always pick the single closest neighbor)

Bias Variance Trade-off

Example:

- **KNN with large $k \rightarrow$ High Bias**
 - Overly simple model
 - high bias (cannot capture structure in the data)
 - low variance (there's enough data to get concentrated estimates)
- **KNN with $k = 1 \rightarrow$ High Variance**
 - Overly complex model
 - Low bias (can learn all relevant structure)
 - High variance (tries to fit all data points)

Two major ensemble techniques

How to reduce bias/variance using ensembling?

- **Bagging:**
 - Reduce variance by averaging the predictions from multiple models.
 - Does not increase bias.
- **Boosting:**
 - Reduce Bias by fitting multiple models which focus on shortcomings of previous models
 - Could potentially increase variance but not by much.

Summary

01

Bias: High bias leads to underfitting, where the model is too simplistic to capture patterns.

02

Variance: High variance leads to overfitting, where the model captures noise along with data patterns.

03

Tradeoff: Reducing bias increases variance and vice versa; the goal is to minimize both.

04

Underfitting: High bias causes low training and test accuracy, indicating a failure to learn from data.

05

Overfitting: High variance results in good training performance but poor generalization to new data.

If you need any help, meet me on **Wednesday**.

Register via **Expert Connect** for this optional technical coaching session.





Quiz

What does a high bias model indicate?

- A** The model fits the training data perfectly but fails on unseen data.
- B** The model is too simple and underfits the training data.
- C** The model is highly sensitive to noise in the training data.
- D** The model generalizes well to new data but performs poorly on the training set.

Quiz

What does a high bias model indicate?

A

The model fits the training data perfectly but fails on unseen data.

B

The model is too simple and underfits the training data.

C

The model is highly sensitive to noise in the training data.

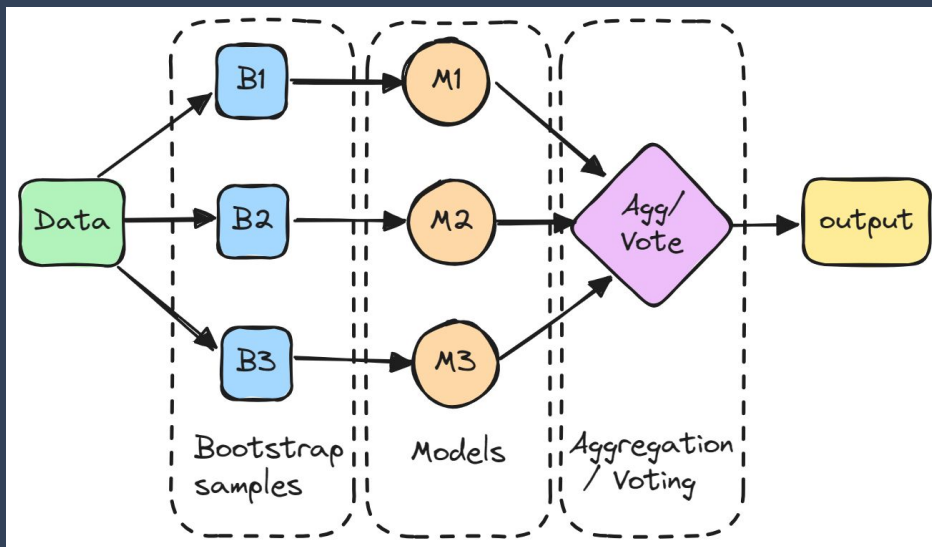
D

The model generalizes well to new data but performs poorly on the training set.

Bagging Algorithms

What is Bagging?

- Bagging stands for **Bootstrap Aggregating**.
- It's an ensemble method that aims to reduce variance and improve model performance by combining multiple models trained on different subsets of the data.



Bagging Intuition

Bagging Process:

- **Random Subsampling** - Create multiple datasets by sampling the original data with replacement.
- **Diversity Injection** - Train each model on different subsets to capture various patterns.

Ensemble Strength:

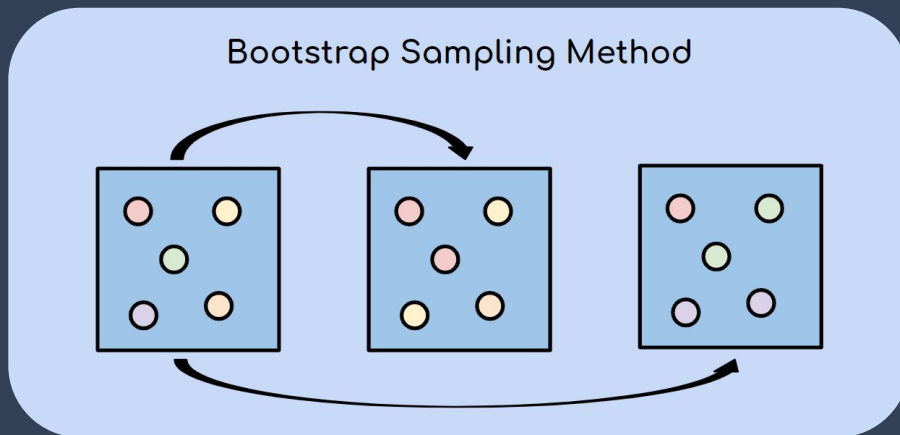
- **Weak Models, Strong Ensemble** - Individually weak models combine to form a powerful predictor.
- **Averaging or Voting** - Final predictions are averaged (regression) or determined by majority vote (classification).

Visual Analogy:

- **Wisdom of Crowds** - Like a diverse group making better decisions together, bagging produces more reliable outcomes by reducing noise.
- **Noise Reduction** - Averaging multiple models filters out noise, resulting in smoother and more accurate predictions.

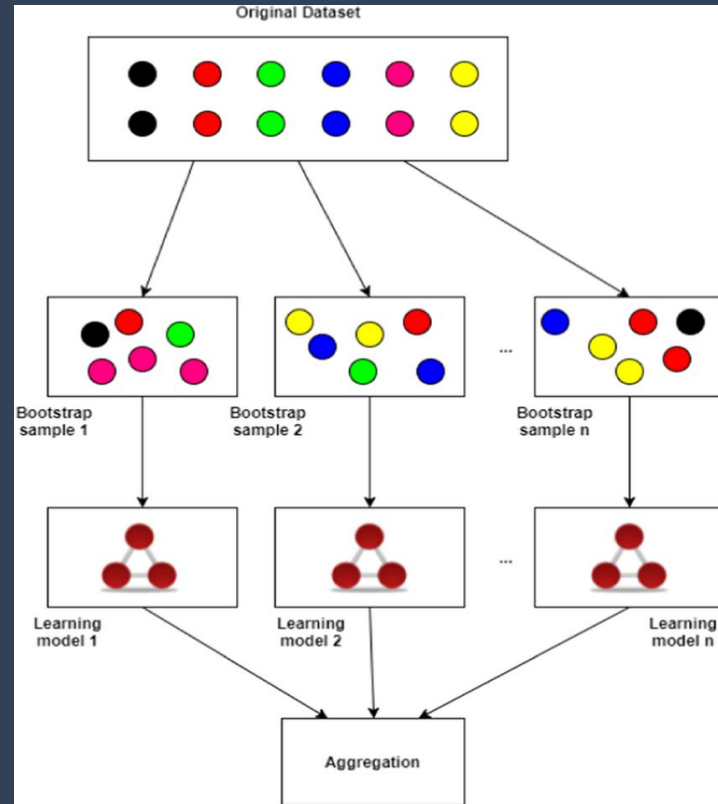
Bootstrapping

- Given training set D with n examples
- Use the empirical distribution P_D as a proxy for P_{sample}
- Basic idea $|D| \rightarrow \text{infinity}$, $P_D \rightarrow P_{\text{Sample}}$
- Generate m new datasets (“resamples” or “bootstrap samples”), each by sampling n training examples from D , **with replacement**



Bootstrap Aggregation

- Once m dataset samples are generated, m models are trained one from each dataset.
- For classification,
 - majority vote from m labels from m Models is considered as final output
 - average of real valued probabilities can be done before thresholding
- For regression, average of m predictions from m models is considered the final output.



Bagging for binary classification

BAGGING

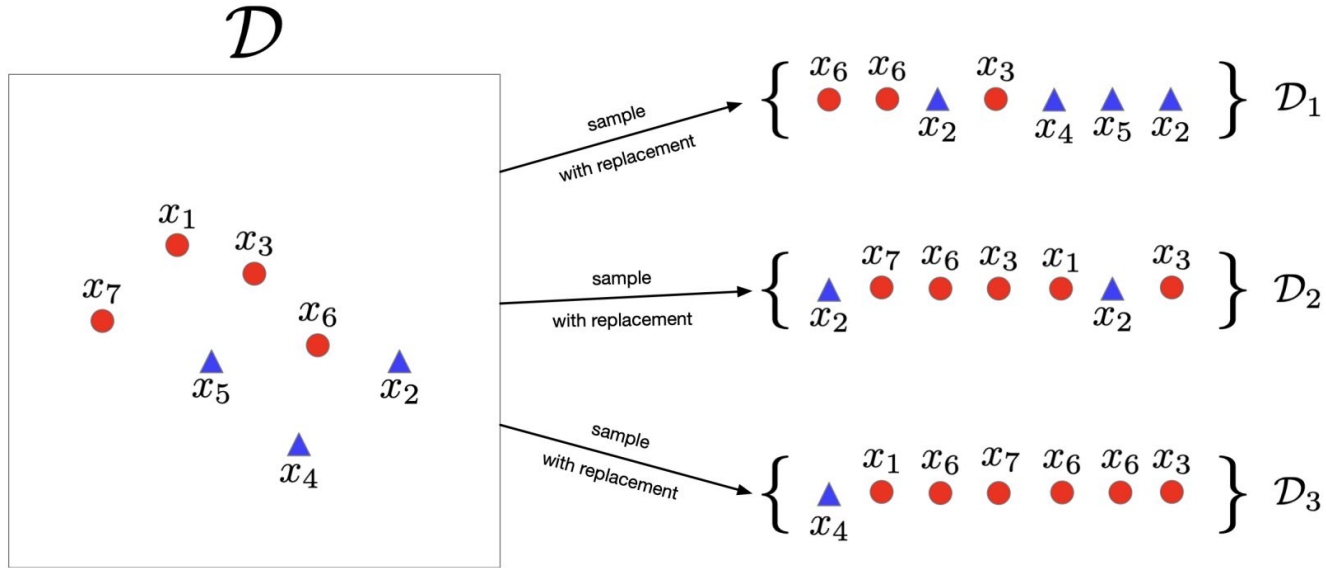
Training phase

1. Initialize the parameters
 - $\mathcal{D} = \emptyset$, the ensemble.
 - L , the number of classifiers to train.
2. For $k = 1, \dots, L$
 - Take a bootstrap sample S_k from $\mathbf{Z}$.
 - Build a classifier D_k using S_k as the training set.
 - Add the classifier to the current ensemble, $\mathcal{D} = \mathcal{D} \cup D_k$.
3. Return $\mathcal{D}$.

Classification phase

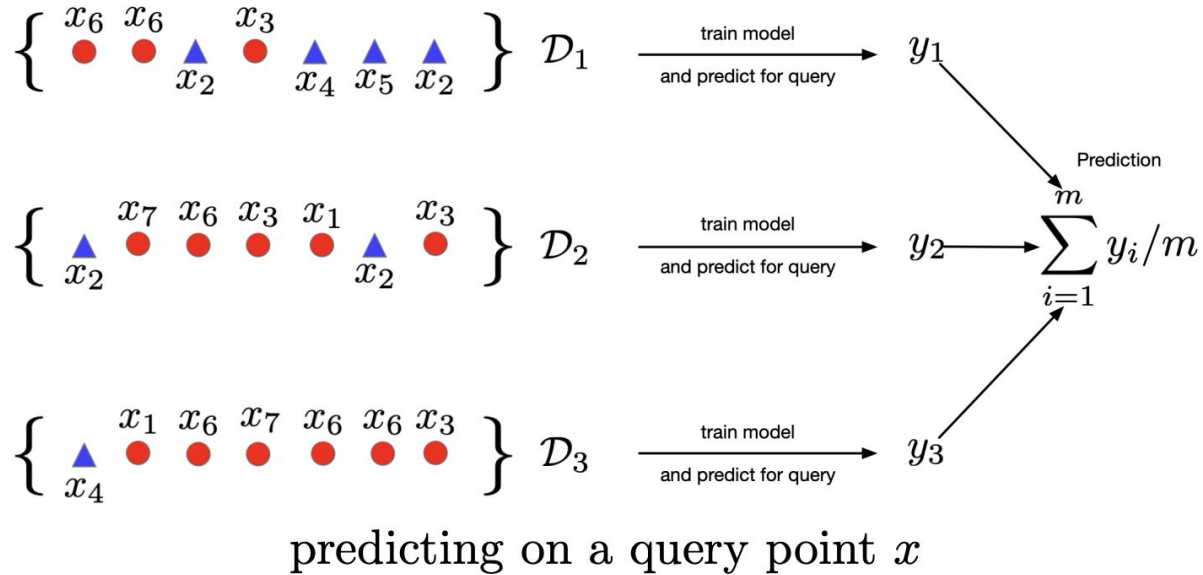
4. Run $D_1, \dots, D_L$ on the input $\mathbf{x}$.
5. The class with the maximum number of votes is chosen as the label for $\mathbf{x}$.

Bagging for Binary Classification



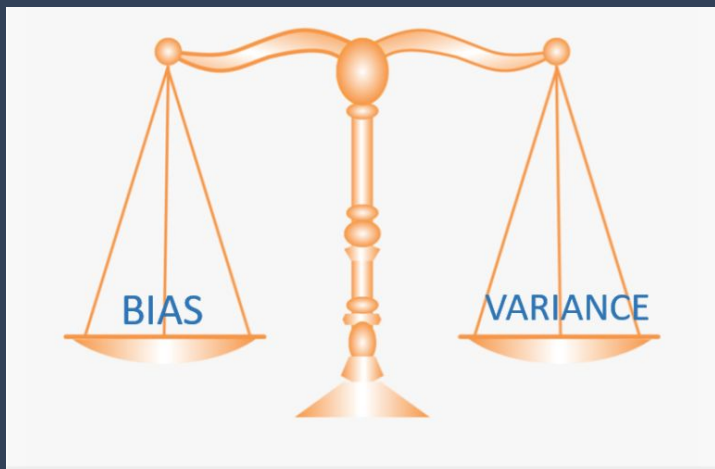
in this example $n = 7, m = 3$

Bagging for Binary Regression



Bias Variance Error Decomposition

$$\begin{aligned}\text{Test MSE} &= \mathbb{E} \left((y - \hat{f}(x))^2 \right) \\ &= \sigma^2 + \left(\text{Bias } \hat{f}(x) \right)^2 + \text{Var} \left(\hat{f}(x) \right)\end{aligned}$$



Bagging - Impact on Bias

How does bagging impact the three terms obtained from bias-variance decomposition?

Bayes error: unchanged, since we have no control over it

Bias: unchanged, since the averaged prediction $\bar{y}$ has the same expectation. $\Rightarrow$ this method will not solve the high bias problem

$$\mathbb{E}[\bar{y}] = \mathbb{E}\left[\frac{1}{m} \sum_{i=1}^m y_i\right] = \mathbb{E}[y_i]$$

Bagging - Impact on Variance

Variance: reduced, since we're averaging over **independent** samples

$$\text{Var}[y] = \text{Var} \left[\frac{1}{m} \sum_{i=1}^m y_i \right] = \frac{1}{m^2} \sum_{i=1}^m \text{Var}[y_i] = \frac{1}{m} \text{Var}[y_i]$$

Bagging: Effect of correlation

- Recall at the beginning, we assumed m independent samples from P_{Sample} to claim $1/m$ variance reduction.
- But since the samples are not independent the actual reduction in variance is different.
- By how much does the variance reduce?
 - It depends on the correlation of the trained models.
- **Q: Does correlation reduce the variance or increases it?**

Bagging: Effect of correlation

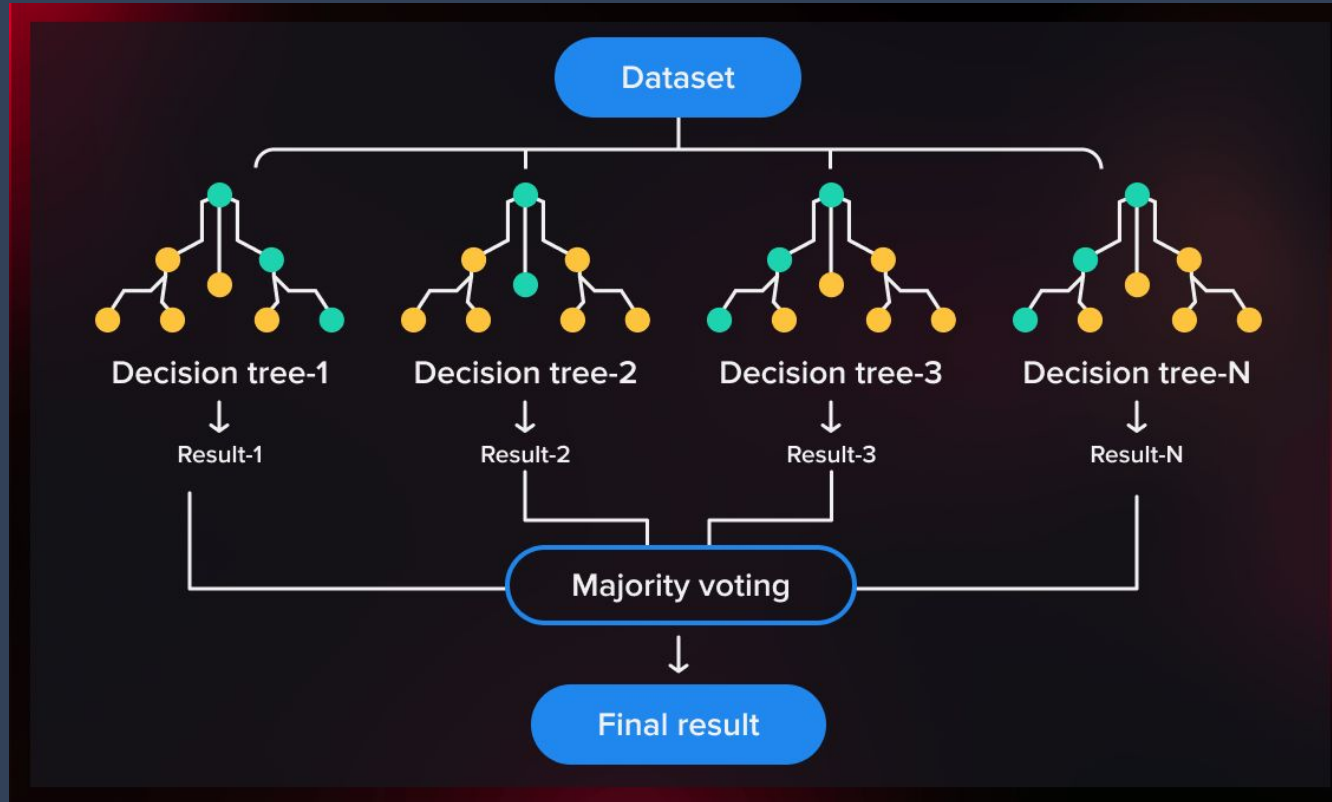
$$\text{Var} \left(\frac{1}{m} \sum_{i=1}^m y_i \right) = \frac{1}{m} (1 - \rho) \sigma^2 + \rho \sigma^2$$

- Bagging is most effective when the individual models are uncorrelated.
- Low correlation between models leads to a greater reduction in variance and improved generalization.
- High correlation between models limits the effectiveness of bagging because the models make similar errors, leading to less variance reduction.

Bagging: Effect of correlation

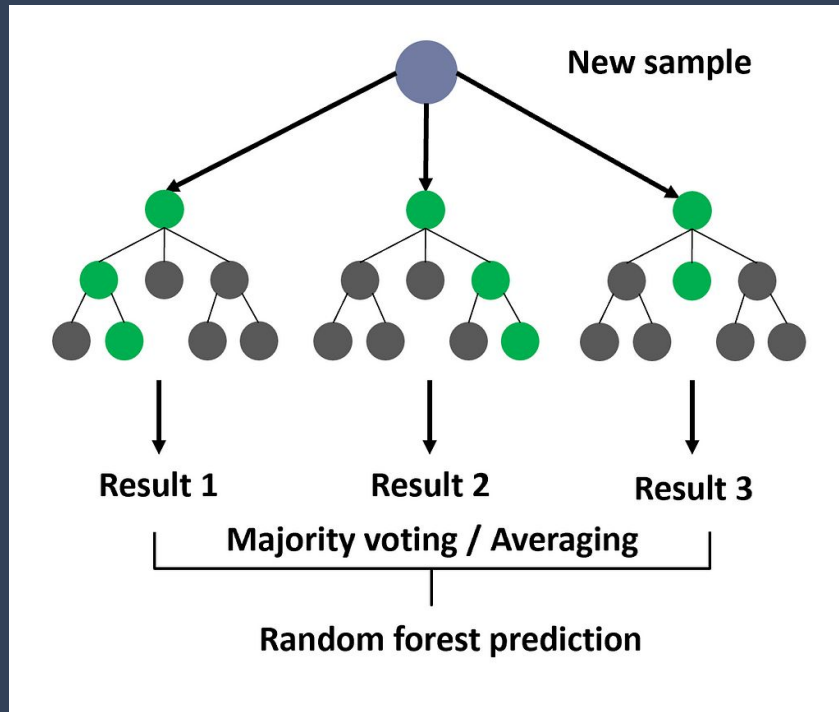
- The variance reduced by factor $1/m$ if correlation is zero.
- The variance is unchanged if correlation is 1.
- Additional variability is good, as it reduces the correlation.
- Multiple algorithms (e.g Decision Tree/ NN/ logistic regression) help.
- Multiple configurations of the same algorithm e.g NN with multiple layer,node combination also helps.
- Reducing correlation in bagging is key.

Random Forests - A Bagging Algorithm



Random Forest

- Bagged decision trees, with one extra trick to de-correlate the predictions
- In each node, choose a random set of k input features, and only consider splits on those features.
- Probably the **best black-box ML algorithm** — they often work well with no tuning.
- Most **widely used** algorithms in Kaggle competitions and industry



Random Forest Algorithm

Algorithm 1: Pseudo code for the random forest algorithm

To generate c classifiers:

for $i = 1$ to c **do**

 Randomly sample the training data D with replacement to produce D_i

 Create a root node, N_i containing D_i

 Call BuildTree(N_i)

end for

BuildTree(N):

if N contains instances of only one class **then**

return

else

 Randomly select $x\%$ of the possible splitting features in N

 Select the feature F with the highest information gain to split on

 Create f child nodes of N , $N_1, \dots, N_f$, where F has f possible values ($F_1, \dots, F_f$)

for $i = 1$ to f **do**

 Set the contents of N_i to D_i , where D_i is all instances in N that match

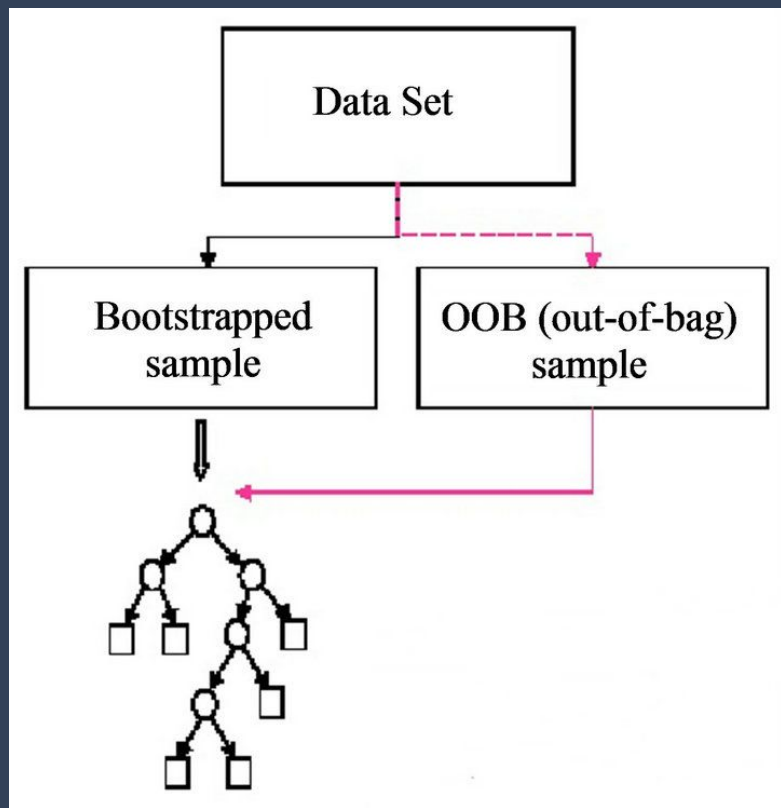
F_i

 Call BuildTree(N_i)

end for

end if

Out of Bag (OOB) in Random Forest



- **Out-of-Bag (OOB) Samples:** Data points that are not included in a particular bootstrap sample used to train an individual tree.
- These OOB samples are then used to test the performance of the tree on data – validation set
- The predictions made by each tree on its respective OOB samples can be averaged to estimate the model's error (OOB error)
- Provides an efficient and unbiased estimate of the model's generalization error

When to use Bagging?

- ✓ High Variance Models
- ✓ Complex Models
- ✓ Ensemble Learning
- ✓ Noisy Data
- ✓ Large and Complex Datasets

.... And when to avoid it

- ✗ Interpretable Models
- ✗ Linear Models
- ✗ Limited Computational Resources
- ✗ Small Datasets
- ✗ Imbalanced Datasets

Overcoming Bagging Limitations

- ✦ **Limited Interpretability:** Utilize model interpretability techniques post-bagging.
- ✦ **Linear Data:** Use Linear Models
- ✦ **Limited Computational Resources:** Optimize bagging's computational demands by parallelizing computations.
- ✦ **Small Datasets:** Increase the size of bootstrap samples to enhance diversity.
- ✦ **Imbalanced Datasets:** Address imbalanced datasets by implementing techniques like stratified sampling during resampling or using specialized bagging variants like Balanced Random Forests.

Bagging - Demo

Demo on Random Forest Algorithm

Summary

01

Bagging : An ensemble method that improves model performance by reducing variance through averaging predictions from multiple models.

02

Bootstrap Sampling: Each model is trained on a random subset of the training data, selected with replacement, creating diverse models.

03

Models are trained independently of one another, which helps reduce overfitting.

04

Most commonly applied in Random Forests, where multiple decision trees are trained using bagging.

05

Out-of-Bag (OOB) Error: A built-in performance evaluation method that uses data points not included in the bootstrap sample for validation.

If you need any help, meet me on **Wednesday**.

Register via **Expert Connect** for this optional technical coaching session.





Quiz

What is a key feature of Random Forest?

- A** Utilizes only one decision tree
- B** Applies bagging to logistic regression
- C** Introduces feature randomization during tree construction
- D** Variables must be scaled before being used for KNN

Quiz

What is a key feature of Random Forest?

A

Utilizes only one decision tree

B

Applies bagging to logistic regression

C

Introduces feature randomization during tree construction

D

Variables must be scaled before being used for KNN



What's ahead?

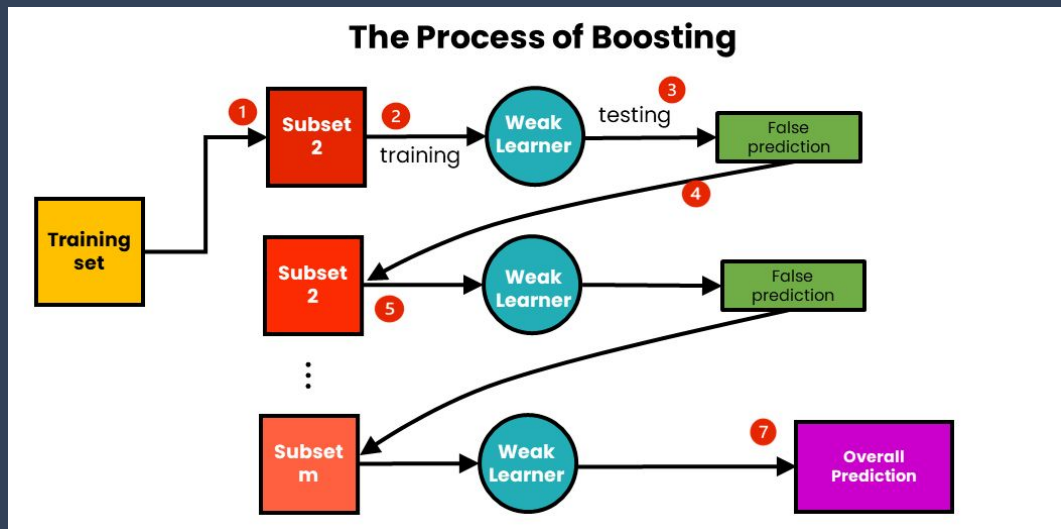


Boosting Algorithms

What is Boosting?

Boosting is an ensemble learning technique that **combines the power of several weak models** to create a strong model.

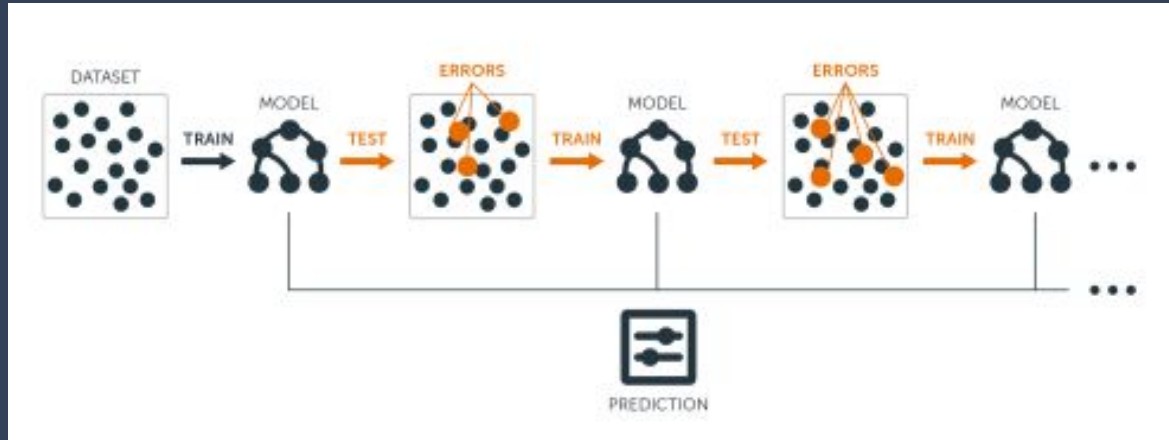
Unlike bagging, where models are built independently, boosting builds models **sequentially**, each one trying to correct the errors of the previous ones.



Boosting: Intuition

- Sequential training of individual models.
- Each time focus on training examples that the previous ones got wrong.
- The shifting focus strongly decorrelates their predictions.

To focus on specific examples, boosting uses a concept of weighted training set

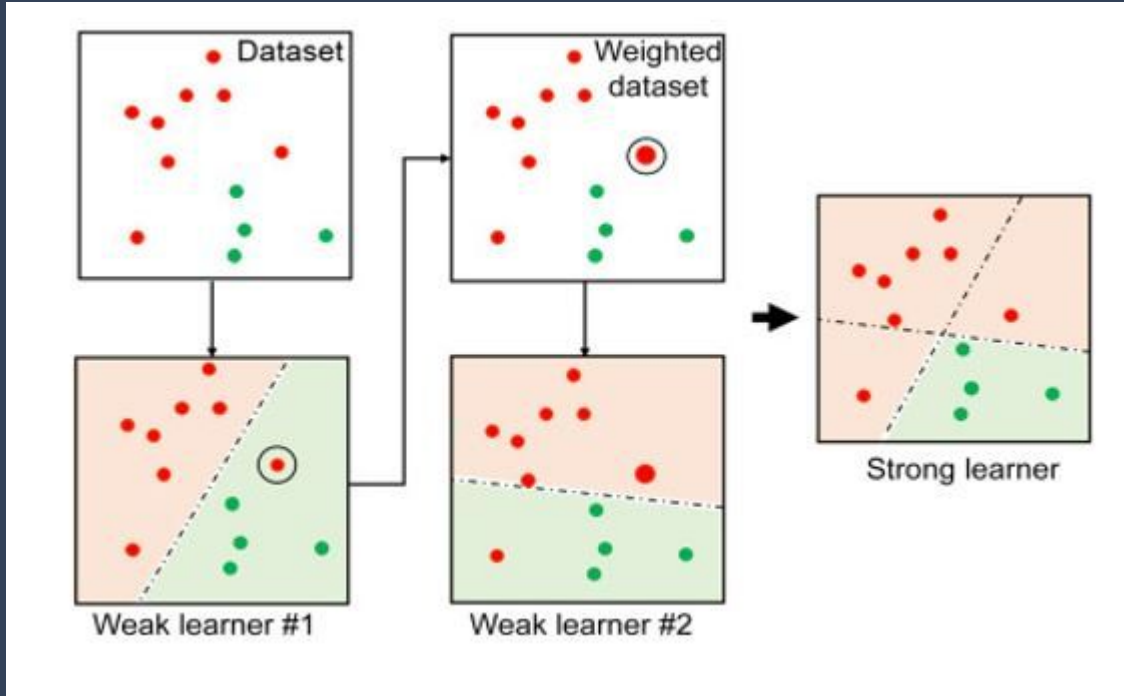


Boosting: Weighted Training Set

- **Key idea:** use different costs (aka weights) for training examples.
- Classifier “**tries harder**” on examples with higher cost. $W^{(n)}$ is the weight assigned to each training example.
- **Weighted Error Rate** is given below

$$\sum_{n=1}^N \frac{1}{N} \mathbb{I}[h(x^{(n)}) \neq t^{(n)}] \quad \text{becomes} \quad \sum_{n=1}^N w^{(n)} \mathbb{I}[h(x^{(n)}) \neq t^{(n)}]$$

AdaBoost - A Boosting algorithm



Weak learner / Strong learner

- Base classifier / Weak learner is a basic ML model that
 1. Is better than a coin toss, i.e. $P[\text{error}] < 0.5$
 2. Super fast during inference
- Strong learner
 - In the PAC learnability sense (Probably Approximately Correct learning)
 - With enough sampling data from P_{Sample} we can make the error go to zero with high probability.

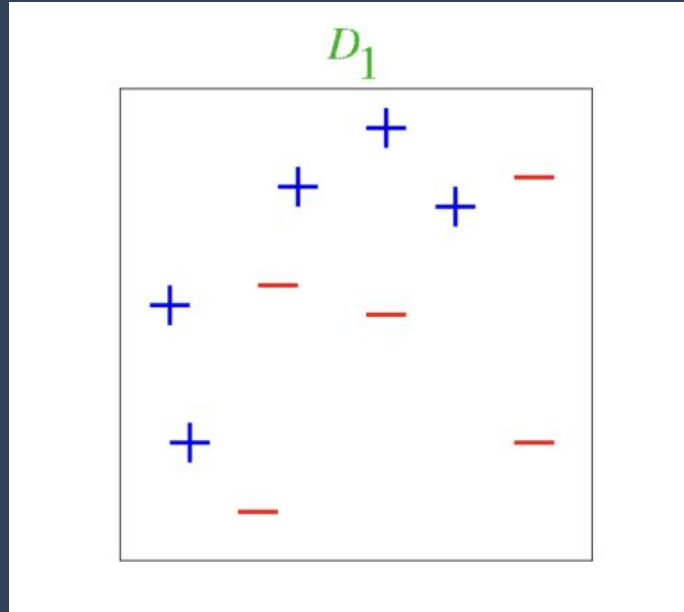
Q: Do you think that it can be made a strong learner?

Weak Learners/Strong Learners

- Adaboost algorithms was a revolution in the 1990s.
- It answered the question in the affirmative: Weak learners can be combined to make strong learners
- Some computationally efficient weak learners are
 - Decision Trees.
 - Decision Stumps (A decision tree with only single split)
- Example strong learners
 - AdaBoost which is based on decision stumps.

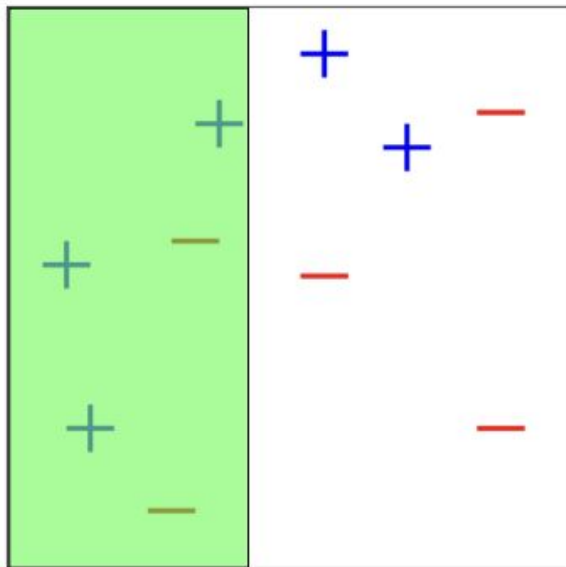
Weak Learners/Classifiers

Q: What is a basic weak learner on this dataset?

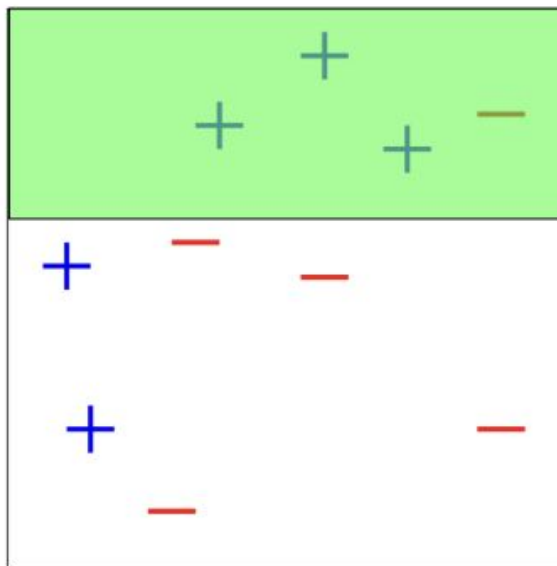


Weak Learners/Classifiers

Vertical half spaces



Horizontal half spaces



Weak Learners/Classifiers

- A single "Weak learner" has high weighted misclassification rate on the training data
- But if a weak learner performs slightly better than chance, i.e. the weighted error of classifier h according to the given weights $w = (w_1, \dots, w_N)$ is at most $0.5 - \gamma$ for some $\gamma > 0$, the AdaBoost algorithm can combine them into a **universal function approximator**.
- An universal function approximator is an algorithm that can pointwise approximate any function under some technical conditions.

Weak Learners/Classifiers

Which feature to split on while learning a decision stump?

- GINI impurity directly chooses the split that minimizes the error.

$$\text{Gini Impurity} = 1 - \sum_{i=1}^n p_i^2$$

AdaBoost - A Boosting algorithm

Algorithm 10.1 *AdaBoost.M1*.

1. Initialize the observation weights $w_i = 1/N$, $i = 1, 2, \dots, N$.
2. For $m = 1$ to M :
 - (a) Fit a classifier $G_m(x)$ to the training data using weights w_i .
 - (b) Compute

$$\text{err}_m = \frac{\sum_{i=1}^N w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^N w_i}.$$

- (c) Compute $\alpha_m = \log((1 - \text{err}_m)/\text{err}_m)$.
 - (d) Set $w_i \leftarrow w_i \cdot \exp[\alpha_m \cdot I(y_i \neq G_m(x_i))]$, $i = 1, 2, \dots, N$.
3. Output $G(x) = \text{sign} \left[\sum_{m=1}^M \alpha_m G_m(x) \right]$.

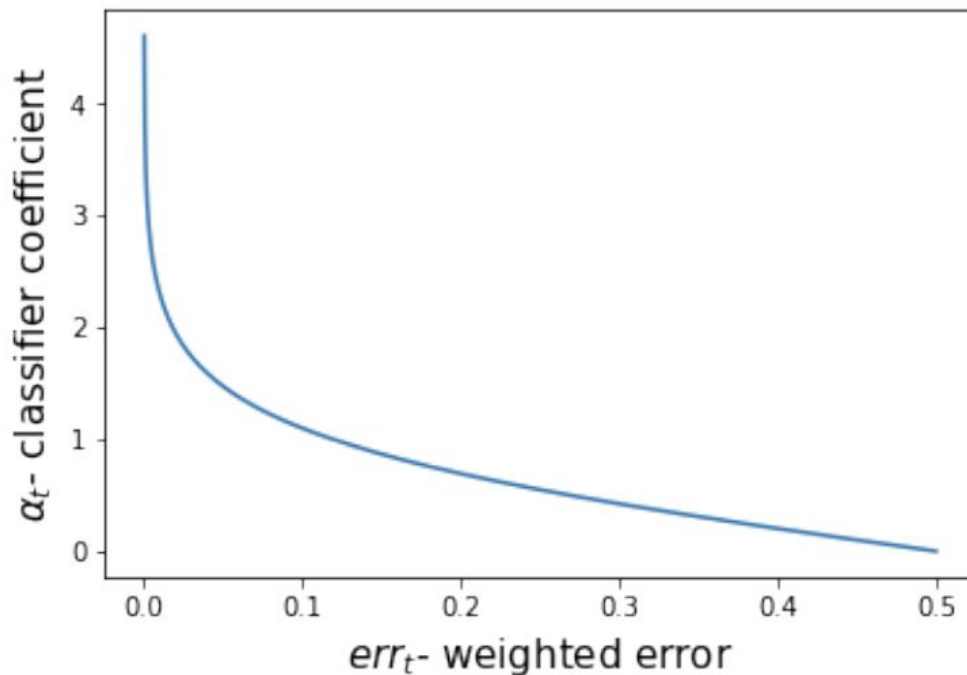
Computing α_t

- Intuition:
 - If classifier $h_t(x)$ is good α_t should be large
 - If classifier $h_t(x)$ is bad α_t should be small
- How to know if $h_t(x)$ is good? - if weighted error is small.
- The loss function that AdaBoost is using is exponential.

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \text{weighted_error}(h_t(x))}{\text{weighted_error}(h_t(x))} \right)$$

Computing α_t

AdaBoost: Relationship between classifier coefficient and weighted error



How to find $w_t^{(i)}$

AdaBoost updates example weights $w_{t+1}^{(i)}$ for iteration $t+1$ based on whether $h_t(x)$ got the example x_i right

- If $h_t(x)$ got x_i **right** then **decrease** the weight $w_{t+1}^{(i)}$

$$w_{t+1}^{(i)} = w_t^{(i)} \cdot \exp(-\alpha_t)$$

- If $h_t(x)$ got x_i **wrong** then **increase** the weight $w_{t+1}^{(i)}$

$$w_{t+1}^{(i)} = w_t^{(i)} \cdot \exp(\alpha_t)$$

How to find $w_t^{(i)}$

- If x_i is often correctly classified by the weak learner then $w^{(i)}$ can go to zero.
- If for x_i the weak learner is often wrong then $w^{(i)}$ can become very large.
- This can introduce numeric instability in the algorithm after some iterations.
- Solution: Normalize the weights so that they add up to 1

$$w_{t+1}^{(i)} = \frac{w_{t+1}^{(i)}}{\sum_{j=1}^N w_{t+1}^{(j)}}$$

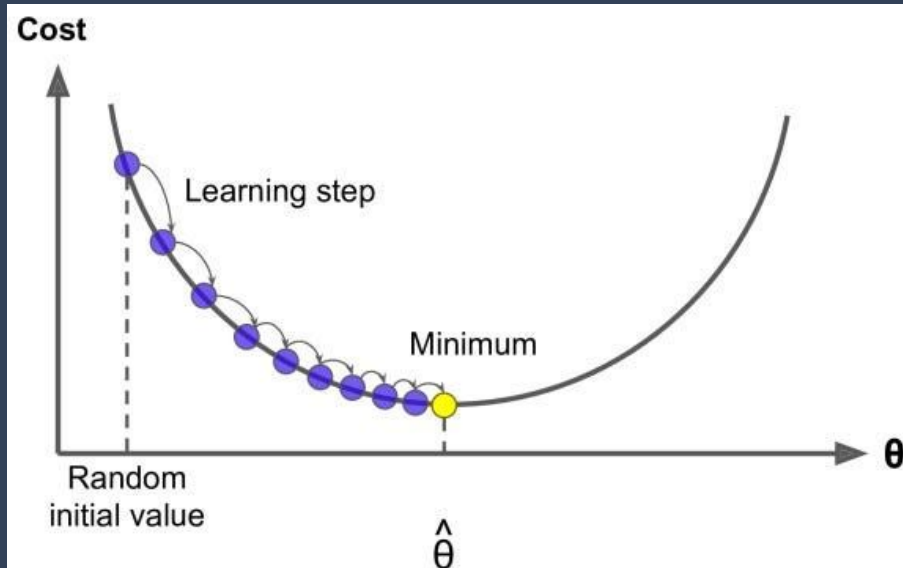
Gradient Boosting: Another example

Gradient Boosting = Gradient Descent + Boosting

1. Sequential ensemble additive model
2. Picks a weak learner that compensates the shortcomings of existing weak learners.
3. Uses gradient descent to move towards the optimal point.
4. In Gradient Boosting, “shortcomings” are identified by gradients.
5. Recall that, in Adaboost, “shortcomings” are identified by high-weight data points.
6. Both high-weight data points and gradients tell us how to improve our model.

What is Gradient Descent?

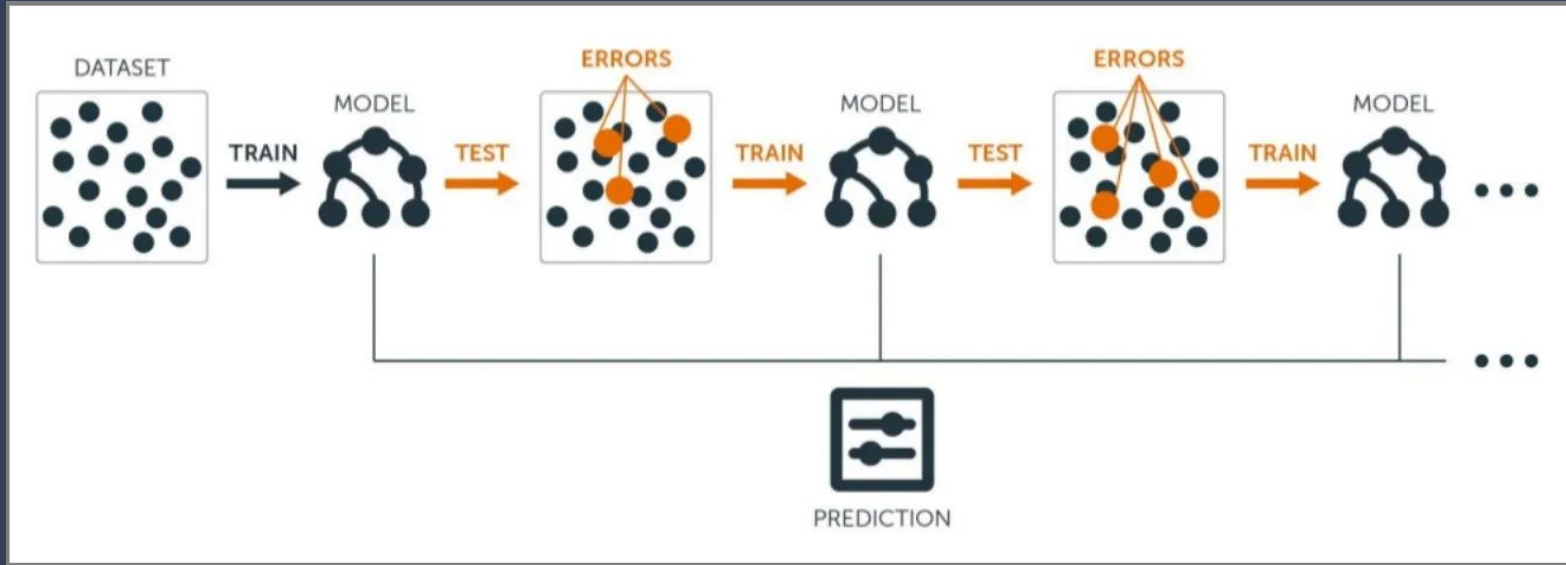
Gradient Descent is an optimization algorithm used in machine learning to minimize the cost (or loss) function, which measures how well a model's predictions match the actual data. Think of it as a method to fine-tune the model so that it makes better predictions.



Note: This will be covered in detail in the later sessions

Intuition

relies on the intuition, that **the best possible next model , when combined with the previous models, minimizes the overall prediction errors.**



Input Requirements

1. **Loss Function**
2. **Weak Learner**
3. **Additive Model** - decision trees are added one at a time (in sequence), and existing trees in the model are not changed.

Gradient Boosting Step by Step -

1. Let's take a dataset

Here, **Age**, **Sft.**, **Location** is independent variables and **Price** is dependent variable or Target variable.

Age	Sft.	Location	Price
5	1500	5	480
11	2030	12	1090
14	1442	6	350
8	2501	4	1310
12	1300	9	400
10	1789	11	500

Gradient Boosting Step by Step -

2. Calculate the average/mean of the target value

$$\frac{480+1090+350+1310+400+500}{6} = 688$$

Age	Sft.	Location	Price	Average_Price
5	1500	5	480	688
11	2030	12	1090	688
14	1442	6	350	688
8	2501	4	1310	688
12	1300	9	400	688
10	1789	11	500	688

Gradient Boosting Step by Step -

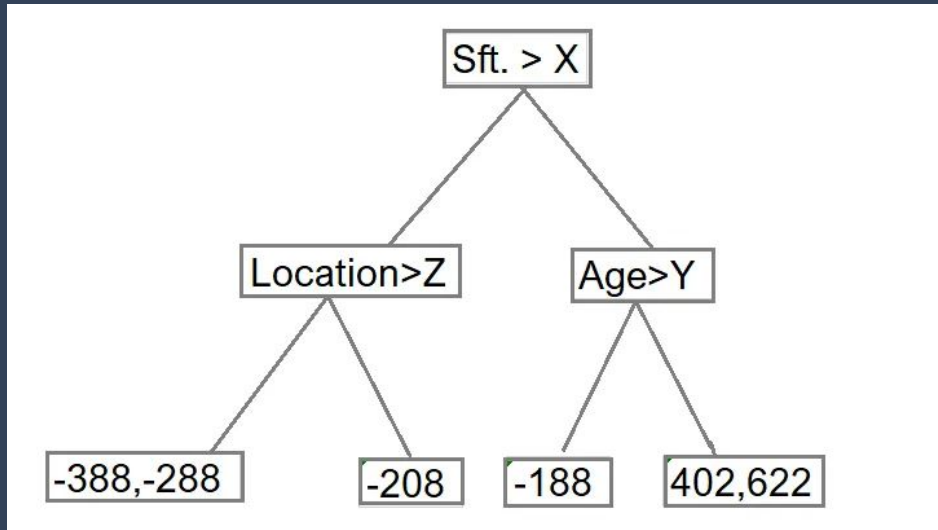
3. Calculate the residuals for each sample.

$$\text{Residual} = \text{Actual Value} - \text{Predicted Value}$$

Age	Sft.	Location	Price	Average_Price	Residual
5	1500	5	480	688	-208
11	2030	12	1090	688	402
14	1442	6	350	688	-338
8	2501	4	1310	688	622
12	1300	9	400	688	-288
10	1789	11	500	688	-188

Gradient Boosting Step by Step -

4. Construct a decision tree. We build a tree with the goal of predicting the Residuals.



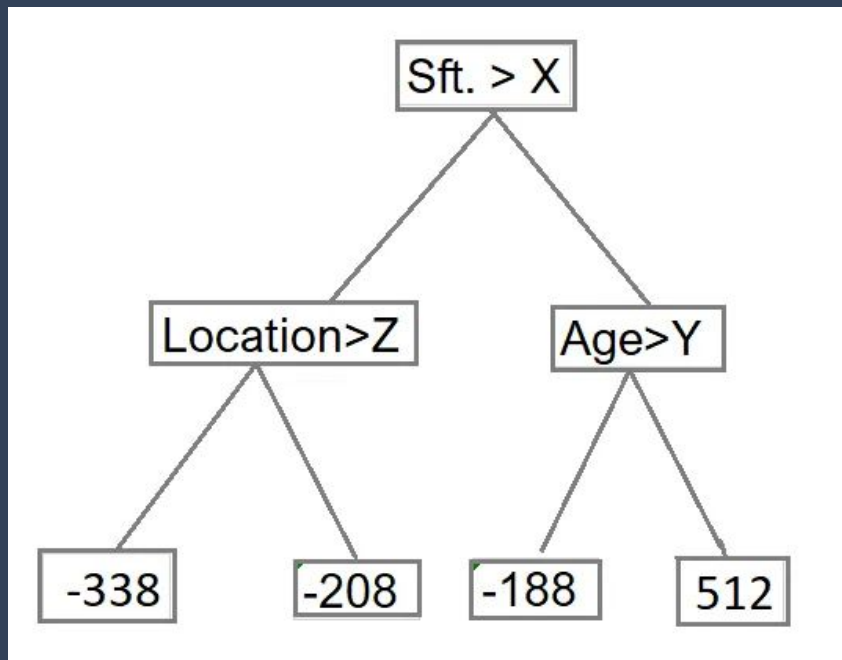
In the event if there are more residuals then leaf nodes(here its 6 residuals),some residuals will end up inside the same leaf. When this happens, we compute their average and place that inside the leaf.

$$\frac{(-388) + (-288)}{2} = -338$$

$$\frac{402 + 622}{2} = 512$$

Gradient Boosting Step by Step -

Now the tree looks like



Gradient Boosting Step by Step -

5. Predict the target label using all the trees within the ensemble.

=>Average Price + Learning Rate * Residual Predicted by decision Tree

$$\Rightarrow 688 + 0.1 * (-338)$$

Calculation for
Residual value (-338)
and (-208) in Step 3

Predicted Price => 654.2

We have initially taken 0.1
as learning rate

$$\Rightarrow 688 + 0.1 * (-208)$$

Predicted Price=> 667.2

Gradient Boosting Step by Step -

6. Compute the new Residuals

Residual=Actual Value - Predicted Value

$$\Rightarrow 350 - 654.2 = -304.2$$

$$\Rightarrow 480 - 667.2 = -187.2$$

When Price is 350 and 480
Respectively. With our Single leaf with
average value(688) we have the below
column of Residual.

Age	Sft.	Location	Price	Average_Price	Residual
5	1500	5	480	688	-208
11	2030	12	1090	688	402
14	1442	6	350	688	-338
8	2501	4	1310	688	622
12	1300	9	400	688	-288
10	1789	11	500	688	-188

Gradient Boosting Step by Step -

With our decision tree ,we ended up the below new residuals.

Age	Sft.	Location	Price	Average_Price	Residual	New Residual
5	1500	5	480	688	-208	-187.2
11	2030	12	1090	688	402	350.8
14	1442	6	350	688	-338	-304.2
8	2501	4	1310	688	622	570.8
12	1300	9	400	688	-288	-254.1
10	1789	11	500	688	-188	-169.2

Gradient Boosting Step by Step -

7. Repeat steps 4 to 6 until the number of iterations matches the number specified by the hyper parameter (numbers of estimators)

8. Once trained, use all of the trees in the ensemble to make a final prediction as to value of the target variable. The final prediction will be equal to the mean we computed in Step 1 plus all the residuals predicted by the trees that make up the forest multiplied by the learning rate.

Here, LR: Learning Rate & DT: Decision Tree

$$\Rightarrow \text{Average Price} + \text{LR} * \text{Residual Predicted by DT 1} + \text{LR} * \text{Residual Predicted by DT 2} + \dots + \text{LR} * \text{Residual Predicted by DT N}$$
$$\Rightarrow 688 + 0.1 * (-188) + 0.1 * (-169.2) + \dots$$

Advantages of Gradient Boosting

- Most of the time predictive accuracy of gradient boosting algorithm on higher side.
- It provides lots of flexibility and can optimize on different loss functions and provides several hyper parameter tuning options that make the function fit very flexible.
- Most of the time no data pre-processing required.
- Gradient Boosting algorithm works great with categorical and numerical data.
- Handles missing data — missing value imputation not required.

Disadvantages of Gradient Boosting

- Gradient Boosting Models will continue improving to minimize all errors. This can overemphasize outliers and cause over fitting. Must use cross-validation to neutralize.
- It is computationally very expensive — GBMs often require many trees (>1000) which can be time and memory exhaustive.
- The high flexibility results in many parameters that interact and influence heavily the behavior of the approach (number of iterations, tree depth, regularization parameters, etc.). This requires a large grid search during tuning.

When to use boosting?

- ✓ High-Quality Predictions
- ✓ Handling Complex Relationships
- ✓ Ensemble of Weak Learners
- ✓ Sequential Adaptation
- ✓ Robustness to Noise

... And when to avoid it

- ✗ Interpretability Requirements
- ✗ Computationally Intensive
- ✗ Small Datasets
- ✗ Linear Relationships
- ✗ Imbalanced Datasets

Overcoming Boosting Limitations

- ✨ **Interpretability Requirement** → Use Interpretable models, PDP
- ✨ **Computationally Intensive** → Reduce complexity / Xgboost or LightGBM
- ✨ **Small Datasets** → Regularisation Hyperparameters
- ✨ **Linear Relationships** → Linear regression/ LASSO
- ✨ **Imbalanced Datasets** → Adjust Class weights

Boosting - Demo

Demo on Boosting Algorithms

Summary

01

Boosting: An ensemble technique that combines multiple weak learners to form a strong learner, improving model accuracy.

02

Boosting trains models sequentially, with each new model correcting errors made by the previous one.

03

Misclassified data points are given more weight, forcing the model to focus on harder cases in subsequent iterations.

04

Popular boosting algorithms include AdaBoost, Gradient Boosting, and XGBoost.

05

The primary aim of boosting is to reduce bias and improve model performance by minimizing errors iteratively.

If you need any help, meet me on **Wednesday**.

Register via **Expert Connect** for this optional technical coaching session.





Quiz

In boosting, what is the primary purpose of the learning rate?

A

Control the number of boosting rounds

B

Adjust the weights of individual weak learners

C

Regulate the speed of convergence during optimization

D

Determine the depth of each weak learner in the ensemble

Quiz - Solution

In boosting, what is the primary purpose of the learning rate?

A

Control the number of boosting rounds

B

Adjust the weights of individual weak learners

C

Regulate the speed of convergence during optimization

D

Determine the depth of each weak learner in the ensemble

Class Summary

Bias - Variance Tradeoff

High bias leads to underfitting, where the model is too simple and misses patterns in the data.

High variance leads to overfitting, where the model is overly sensitive to noise in the training data.

The goal is to balance bias and variance for optimal model performance, reducing overall error.

Bagging

Bagging reduces variance by averaging predictions from multiple models trained on different data subsets.

Models are trained on random subsets of data with replacement, creating diverse models - Bootstrap Sampling.

Each model is trained independently, improving stability and reducing overfitting.

Boosting

Boosting builds models sequentially, with each new model focusing on correcting the errors of the previous one.

Misclassified data points get higher weights, forcing the model to focus on harder cases.

Boosting primarily aims to reduce bias by improving model accuracy iteratively.



End of the Class Checklist - For Instructors

- ❑ Have you shared the the following with the Ops team or in the discord channel?
 - ❑ Annotated Slides
 - ❑ Colab notebook if you made any changes for this class
 - ❑ Reference Materials if you used/ mentioned any during the class
- ❑ Have you encouraged the learners to attempt the assignment after the class?
- ❑ Have you encouraged the learners to utilize the Wednesday session for clarifying their queries & Thursday session for assignment walkthrough?